

Guide de Deploiement — GMAO MVP

Objectif : Deployer l'app pour qu'elle soit testable en ligne via GitHub. **Contexte :** MVP d'apprentissage. Deploiement simple, pas de DevOps complexe. **Repo GitHub requis :** ton code doit etre pousse sur un repo GitHub

Comprendre le deploiement avant de commencer

C'est quoi "deployer" ?

Aujourd'hui ton app tourne sur ton ordinateur (`localhost:8000`). Personne d'autre ne peut y acceder. **Deployer** = mettre ton code sur un serveur distant pour que n'importe qui puisse y acceder via une URL (ex: `https://gmao.mondomaine.com`).

Ce dont ton app Symfony a besoin pour tourner

Besoin	Pourquoi	En local	En production
PHP 8.4	Symfony est ecrit en PHP	Installe sur ton Mac	Fourni par l'hebergeur
MySQL 8	Stocker les donnees	Ton MySQL local	Base de donnees de l'hebergeur
Composer	Installer les dependances PHP	Installe sur ton Mac	A executer sur le serveur
Un serveur web	Recevoir les requetes HTTP	<code>symfony server:start</code>	Apache ou Nginx (fourni par l'hebergeur)
HTTPS	Securiser les echanges	Pas necessaire en local	Certificat SSL (gratuit avec Let's Encrypt)
Stockage fichiers	Photos uploadées	<code>var/uploads/photos/</code>	Un dossier sur le serveur

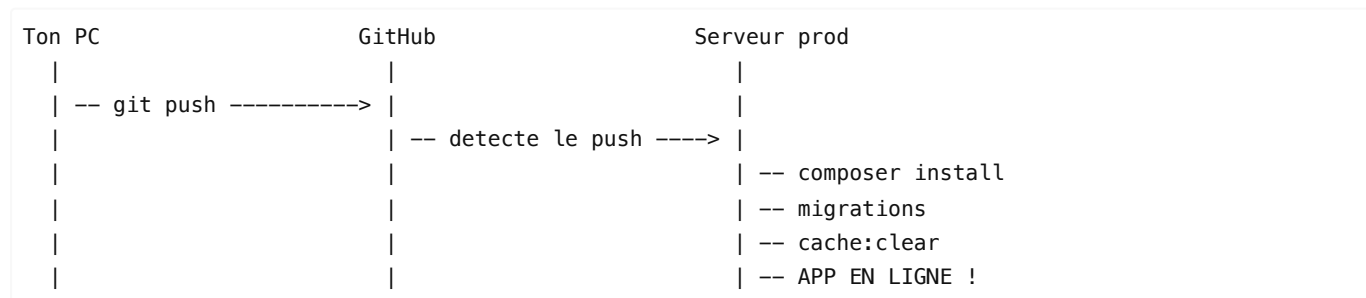
C'est quoi le fichier `.env.local` ?

Ton app a besoin de parametres qui changent selon l'environnement :

- En local : `DATABASE_URL=mysql://root:@127.0.0.1:3306/gmao`
- En production : `DATABASE_URL=mysql://user_prod:mdp_fort@serveur_prod:3306/gmao_prod`

Le fichier `.env` contient les valeurs par default (dev). Le fichier `.env.local` contient les valeurs specifiques a l'environnement (prod). `.env.local` n'est JAMAIS commite dans Git (il est dans `.gitignore`).

Le flux de deploiement



Pourquoi PAS Vercel ?

Tu as mentionné Vercel. C'est une excellente plateforme, mais **pas adaptée pour Symfony** :

- Vercel est conçu pour Next.js, React, et les sites statiques
- Le support PHP est expérimental et très limité
- Pas de base de données MySQL intégrée
- Pas de stockage de fichiers persistant (les photos uploadées seraient perdues)
- Les sessions PHP ne fonctionnent pas normalement

Conclusion : Vercel = excellent pour du front (React, Next.js), mauvais pour du Symfony.

Option 1 : AlwaysData (RECOMMANDE pour ton cas)

Pourquoi AlwaysData ?

- **Hebergeur français** (support en français, serveurs en France)
- **Offre gratuite** : 100 Mo de stockage + 1 base MySQL (parfait pour tester le MVP)
- **Support natif PHP 8.4** + MySQL 8
- **SSH disponible** pour exécuter les commandes Symfony
- **Git installé** sur le serveur → tu peux cloner ton repo directement
- **Déploiement via Git** : tu pushes sur GitHub, puis un `git pull` sur le serveur
- **Interface simple** pour configurer PHP, les bases de données, etc.

Etape par étape

Etape 1 : Créer un compte AlwaysData

1. Aller sur **alwaysdata.com**
2. Cliquer "Essai gratuit" (pas besoin de carte bancaire)
3. Choisir un nom de compte (ex: `diakite-gmao`)
4. Tu reçois un sous-domaine gratuit : `diakite-gmao.alwaysdata.net`

Ce que tu obtiens :

- Un espace d'hébergement avec SSH
 - Une base MySQL
 - Un PhpMyAdmin
 - Un sous-domaine HTTPS gratuit
-

Etape 2 : Créer la base de données MySQL

1. Dans le panneau AlwaysData : **Bases de données > MySQL**
2. Cliquer "**Ajouter une base de données**"
3. Nom : `gmao` (le nom complet sera `diakite-gmao_gmao`)
4. AlwaysData crée automatiquement un utilisateur MySQL avec le même nom que ton compte

Note les informations :

```
Hôte      : mysql-diakite-gmao.alwaysdata.net
Base      : diakite-gmao_gmao
User      : diakite-gmao
Password  : (celui de ton compte AlwaysData)
```

Etape 3 : Configurer PHP

1. Dans le panneau : **Environnement > PHP**
 2. Version PHP : **8.4**
 3. Extensions a activer (si pas deja actives) :
 - `intl` (internationalisation, requis par Symfony)
 - `pdo_mysql` (connexion MySQL)
 - `mbstring` (gestion des caracteres UTF-8)
 - `gd` (traitement d'images)
 - `zip` (requis par Composer)
-

Etape 4 : Se connecter en SSH et cloner le projet

C'est quoi SSH ? SSH = un terminal distant. Tu tapes des commandes comme si tu etais sur le serveur.

1. Ouvrir ton terminal (Mac/Linux) :

```
ssh diakite-gmao@ssh-diakite-gmao.alwaysdata.net
```

Mot de passe : celui de ton compte AlwaysData

2. Tu es maintenant connecte au serveur. Verifie :

```
php -v          # Doit afficher PHP 8.4.x
composer -V     # Doit afficher Composer 2.x
git --version   # Doit afficher git 2.x
```

3. **Aller dans le dossier web :**

```
cd ~/www
```

Ce dossier `~/www` est la racine web. Tout ce qui est dedans est accessible via le navigateur.

4. **Supprimer le contenu par default :**

```
rm -rf ~/www/*
```

5. **Cloner ton repo GitHub :**

```
git clone https://github.com/TON_USER/GMA0.git ~/www
```

Remplace `TON_USER` par ton nom d'utilisateur GitHub.

Si ton repo est prive, tu auras besoin d'un token GitHub :

```
git clone https://TON_USER:TON_TOKEN@github.com/TON_USER/GMA0.git ~/www
```

Pour generer un token : GitHub > Settings > Developer settings > Personal access tokens > Generate new token (cocher `repo`)

Etape 5 : Installer les dependances

```
cd ~/www
```

```
# Installer les dependances PHP (SANS les packages de developpement)
composer install --no-dev --optimize-autoloader
```

Pourquoi `--no-dev` ? En production, on n'a pas besoin de PHPUnit, des fixtures, du profiler Symfony, etc. Ca reduit la taille et ameliore la securite.

Pourquoi `--optimize-autoloader` ? Ca cree une "carte" de toutes les classes PHP pour que Symfony les trouve instantanement au lieu de chercher dans les dossiers.

Etape 6 : Creer le fichier .env.local

```
cp .env .env.local
nano .env.local
```

Contenu a ecrire :

```
# Mode production (desactive le profiler, le debug, les messages d'erreur detailles)
APP_ENV=prod

# Cle secrete unique pour ton app (securite CSRF, cookies, etc.)
# GENERE CETTE CLE TOI-MEME, ne copie pas celle-ci :
APP_SECRET=a1b2c3d4e5f6789012345678abcdef00

# Connexion a la base de donnees AlwaysData
DATABASE_URL="mysql://diakite-gmao:TON_MOT_DE_PASSE@mysql-diakite-gmao.alwaysdata.net:3306/diakite-gmao_gmao?serverVersion=8.0"

# Mailer (pour l'instant on desactive, on configurera plus tard)
MAILER_DSN=null://null
```

Pour generer APP_SECRET :

```
php -r "echo bin2hex(random_bytes(16)) . PHP_EOL;"
```

Copie le resultat et colle-le a la place de `a1b2c3d4e5f6789012345678abcdef00`.

Sauvegarder dans nano : `Ctrl+O` puis `Entree` puis `Ctrl+X`

Etape 7 : Creer les tables en base de donnees

```
# Executer toutes les migrations (cree les tables)
php bin/console doctrine:migrations:migrate --no-interaction
```

Ce que ca fait : Chaque fichier dans `migrations/` contient des instructions SQL. Cette commande les execute dans l'ordre pour creer toutes tes tables (organisation, user, site, demande, etc.).

Optionnel — charger les donnees de demo :

```
php bin/console doctrine:fixtures:load --append
```

Etape 8 : Vider le cache et optimiser

```
# Vider le cache de dev (s'il existe) et construire le cache de prod
php bin/console cache:clear --env=prod

# Pre-construire le cache (plus rapide au premier chargement)
php bin/console cache:warmup --env=prod
```

Pourquoi ? Symfony compile les templates Twig, les routes, les services... en fichiers de cache. En prod, tout est pre-compile pour aller plus vite.

Etape 9 : Configurer le Document Root

C'est quoi le Document Root ? C'est le dossier que le serveur web expose au navigateur. Pour Symfony, ce doit être le dossier `public/` (pas la racine du projet).

1. Dans le panneau AlwaysData : **Sites > Modifier**
2. Champ "**Repertoire**" : mettre `/www/public/`
3. Type : **PHP**
4. Sauvegarder

Pourquoi `/www/public/` ? Le dossier `public/` contient `index.php` (le point d'entrée de Symfony). Les dossiers `src/`, `config/`, `var/` ne doivent JAMAIS être accessibles depuis le navigateur (sécurité).

Etape 10 : Créer le dossier d'upload

```
# Créer le dossier pour les photos
mkdir -p ~/www/var/uploads/photos

# Donner les droits d'écriture
chmod -R 775 ~/www/var
```

Etape 11 : Tester

1. Ouvrir `https://diakite-gmao.alwaysdata.net` dans ton navigateur
2. Tu devrais voir la page de login
3. Se connecter avec `admin@gmao.fr / Test1234!`
4. Naviguer dans l'app, créer une demande, tester les photos

Si tu vois une erreur 500 :

```
# Regarder les logs Symfony
tail -50 ~/www/var/log/prod.log
```

Etape 12 : Mettre à jour après un push GitHub

Quand tu fais des modifications en local et que tu pushes sur GitHub :

```
# 1. Se connecter en SSH
ssh diakite-gmao@ssh-diakite-gmao.alwaysdata.net

# 2. Aller dans le projet
cd ~/www
```

```
# 3. Recuperer les derniers changements
git pull origin main

# 4. Installer les nouvelles dependances (si composer.json a change)
composer install --no-dev --optimize-autoloader

# 5. Executer les nouvelles migrations (si il y en a)
php bin/console doctrine:migrations:migrate --no-interaction

# 6. Vider le cache
php bin/console cache:clear --env=prod
```

Astuce : Tu peux creer un script `deploy.sh` pour automatiser tout ca :

```
#!/bin/bash
cd ~/www
git pull origin main
composer install --no-dev --optimize-autoloader
php bin/console doctrine:migrations:migrate --no-interaction
php bin/console cache:clear --env=prod
echo "Deploiement termine !"
```

Puis lancer : `bash ~/deploy.sh`

Option 2 : Fly.io (plus moderne, deploy automatique)

Pourquoi Fly.io ?

- **Deploy automatique depuis GitHub** (push = deploy)
- **Gratuit** pour commencer (3 machines gratuites)
- **Utilise Docker** : tu decris ton environnement dans un fichier, Fly.io le construit
- **Serveurs partout dans le monde** (tu choisis Paris)
- **Base de donnees MySQL** via un add-on ou un service externe

Comment ca marche ?

Fly.io fonctionne avec des **conteneurs Docker**. C'est comme une "boite" qui contient :

- PHP 8.4
- Ton code Symfony
- Toutes les dependances
- La config Nginx

Tu decris cette boite dans un fichier `Dockerfile` . Fly.io construit la boite et la lance sur un serveur.

Etape par etape

Etape 1 : Installer Fly CLI

```
# Sur Mac
brew install flyctl
```

```
# Se connecter (cree un compte si necessaire)
fly auth login
```

Etape 2 : Créer le Dockerfile

C'est quoi un Dockerfile ? C'est une recette qui dit à Fly.io comment construire l'environnement de ton app.

Créer `Dockerfile` à la racine du projet :

```
# Partir d'une image PHP officielle avec Apache
FROM php:8.4-apache

# Installer les extensions PHP necessaires
RUN apt-get update && apt-get install -y \
    libicu-dev libzip-dev libpng-dev libjpeg-dev unzip git \
    && docker-php-ext-configure gd --with-jpeg \
    && docker-php-ext-install intl pdo_mysql zip gd opcache

# Activer le module Apache rewrite (necessaire pour Symfony)
RUN a2enmod rewrite

# Installer Composer
COPY --from=composer:2 /usr/bin/composer /usr/bin/composer

# Configurer Apache pour pointer vers /public
ENV APACHE_DOCUMENT_ROOT=/var/www/html/public
RUN sed -ri 's!/var/www/html!${APACHE_DOCUMENT_ROOT}!g' /etc/apache2/sites-available/000-default.conf

# Copier le code du projet
COPY . /var/www/html/

# Installer les dependances
RUN composer install --no-dev --optimize-autoloader

# Créer le dossier d'upload et donner les droits
RUN mkdir -p var/uploads/photos \
    && chown -R www-data:www-data var/

# Exposer le port 8080 (requis par Fly.io)
EXPOSE 8080

# Configurer Apache sur le port 8080
RUN sed -i 's/80/8080/g' /etc/apache2/ports.conf /etc/apache2/sites-available/000-default.conf
```

Etape 3 : Créer le fichier fly.toml

```
fly launch --no-deploy
```

Fly.io genere un fichier `fly.toml` . Le modifier :

```
app = "gmao-mvp"
primary_region = "cdg" # Paris

[build]
```

```
[http_service]
  internal_port = 8080
  force_https = true

[env]
  APP_ENV = "prod"

[[vm]]
  size = "shared-cpu-1x"
  memory = "512mb"
```

Etape 4 : Configurer les secrets (variables sensibles)

```
# Ces commandes stockent les variables de maniere securisee (pas dans le code)
fly secrets set APP_SECRET=$(php -r "echo bin2hex(random_bytes(16));")
fly secrets set DATABASE_URL="mysql://user:password@host:3306/gmao?serverVersion=8.0"
fly secrets set MAILER_DSN="null://null"
```

Pourquoi des "secrets" ? Les mots de passe et cles ne doivent JAMAIS etre dans le code ou dans Git. Fly.io les injecte comme variables d'environnement au demarrage du conteneur.

Etape 5 : Base de donnees

Option A — PlanetScale (MySQL gratuit dans le cloud) :

1. Creer un compte sur planetscale.com
2. Creer une base "gmao"
3. Recuperer l'URL de connexion
4. `fly secrets set DATABASE_URL="mysql://..."`

Option B — Fly Postgres (gratuit, mais PostgreSQL pas MySQL) :

```
fly postgres create --name gmao-db
fly postgres attach gmao-db
```

Necessite de changer `pdo_mysql` par `pdo_pgsql` dans le Dockerfile (pas recommande si tu veux rester sur MySQL).

Etape 6 : Deployer

```
fly deploy
```

Cette commande :

1. Envoie ton code a Fly.io
 2. Construit le conteneur Docker
 3. Demarre l'app sur un serveur a Paris
 4. Te donne l'URL : `https://gmao-mvp.fly.dev`
-

Etape 7 : Executer les migrations

```
# Ouvrir un terminal sur le conteneur distant
fly ssh console

# A l'interieur du conteneur :
cd /var/www/html
php bin/console doctrine:migrations:migrate --no-interaction
php bin/console doctrine:fixtures:load --append # optionnel
```

Etape 8 : Deploy automatique via GitHub Actions

Creer `.github/workflows/deploy.yml` :

```
name: Deploy to Fly.io

on:
  push:
    branches: [main]

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: superfly/flyctl-actions/setup-flyctl@master
      - run: flyctl deploy --remote-only
    env:
      FLY_API_TOKEN: ${ secrets.FLY_API_TOKEN }
```

Ajouter le token Fly.io dans GitHub :

1. `fly tokens create deploy`
2. GitHub > Settings > Secrets > New secret > `FLY_API_TOKEN`

Resultat : Chaque `git push` sur `main` deploie automatiquement.

Option 3 : AlwaysData avec deploy automatique (GitHub Actions)

Si tu choisis AlwaysData mais que tu veux aussi le deploy automatique comme Fly.io :

Creer `.github/workflows/deploy.yml` :

```
name: Deploy to AlwaysData

on:
  push:
    branches: [main]

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Deploy via SSH
        uses: appleboy/ssh-action@v1
```

```
with:
  host: ssh-diakite-gmao.alwaysdata.net
  username: diakite-gmao
  password: ${ secrets.ALWAYS_DATA_PASSWORD }
  script: |
    cd ~/www
    git pull origin main
    composer install --no-dev --optimize-autoloader
    php bin/console doctrine:migrations:migrate --no-interaction
    php bin/console cache:clear --env=prod
```

Ajouter le secret dans GitHub :

1. GitHub > Repo > Settings > Secrets > New secret
2. Nom : ALWAYS_DATA_PASSWORD , Valeur : ton mot de passe AlwaysData

Resultat : `git push` → GitHub Actions se connecte en SSH → execute le deploy.

Comparatif final

Critere	AlwaysData	Fly.io
Prix	Gratuit (100 Mo) puis 2€/mois	Gratuit (limites) puis ~3\$/mois
Langue	Francais	Anglais
Facilite	Tres simple (hebergement classique)	Moyenne (Docker a comprendre)
Deploy auto GitHub	Oui (avec GitHub Actions)	Oui (natif + GitHub Actions)
PHP 8.4	Oui (natif)	Oui (via Docker)
MySQL	Oui (inclus)	Non inclus (PlanetScale ou autre)
SSH	Oui	Oui (fly ssh console)
HTTPS	Oui (automatique)	Oui (automatique)
Upload fichiers	Oui (persistant)	Attention : les fichiers sont perdus a chaque redeploy
Stockage	100 Mo gratuit	Besoin d'un volume (\$)
Ideal pour	MVP simple, apprendre le deploiement	App conteneurisee, approche moderne

Ma recommandation

Pour ton cas (MVP + apprentissage + simplicité) : AlwaysData

Raisons :

1. **Gratuit** pour tester
2. **Francais** (support, interface, documentation)
3. **PHP natif** (pas de Docker a apprendre en plus)
4. **MySQL inclus** (pas de service externe a configurer)
5. **Upload de fichiers persistant** (les photos restent entre les deploys)
6. **SSH simple** (comme un terminal distant)

7. Deploy auto possible avec GitHub Actions

Quand passer a Fly.io ou un VPS ? Quand ton app aura de vrais utilisateurs et que tu auras besoin de plus de puissance ou de conteneurisation.

Checklist pre-deploiement

Avant de deployer, verifie que tout est OK :

- ☐ Code pousse sur GitHub (`git push origin main`)
- ☐ `.env.local` est dans `.gitignore` (JAMAIS commite)
- ☐ `var/` est dans `.gitignore`
- ☐ `vendor/` est dans `.gitignore`
- ☐ Les migrations sont a jour (`php bin/console doctrine:migrations:status`)
- ☐ `php bin/console lint:container` ne retourne aucune erreur
- ☐ `php bin/console lint:twig templates/` valide tous les templates

Apres le deploiement :

- ☐ La page de login s'affiche (`https://ton-sous-domaine.alwaysdata.net`)
- ☐ Login avec `admin@gmao.fr` / `Test1234!` fonctionne
- ☐ Le dashboard s'affiche avec les KPI
- ☐ Upload d'une photo fonctionne
- ☐ Les filtres sur les demandes fonctionnent
- ☐ HTTPS actif (cadenas vert dans le navigateur)